

# Semester Recap for Theoretical Computer Science

by Erik Girdawskas

color code: Topic / Theorem / Lemma / Observation / other important things

## Preface:

- DiskMath may play a role in this? I see a bit in the 1st Serie already
    - exhibit A: Cantor's Diagonalization: remember how one common disproof of  $\mathbb{N}$ 's countability goes: "imagine a table of all real numbers (for simplification, only between 0 and 1"  $\Rightarrow$  consider a number that at decimal point  $i$  has a number different to what the  $i$ -th number in the table has at decimal point  $i \Rightarrow$  this number cannot be in the table  $\Rightarrow$  there cannot be such a table  $\Rightarrow \mathbb{R}$  is uncountable
    - I will now go into some abstract theory, then apply the diagonalization
    - basically, an alphabet is a set of symbols and a word is a sequence of symbols
    - for example,  $\text{DEZ} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ ,  $x = 1381$  is a word in DEZ
    - an Entscheidungsproblem (the actual academic term is in German!) (abbreviated to EP) is the problem of deciding whether a word  $x$  is in the alphabet  $\_$  and has a binary answer
    - for example, consider the alphabet  $\text{PRIM} = \{y \mid y \text{ is prime}\}$  with  $y \in \text{DEZ}$ , is some word  $x \in \text{PRIM}$  (EP)
    - Imagine a perfect mathematician with infinite pen/paper/rules (for derivations), their input is a sequence of symbols (a word) on paper, they independently produce an output that is also a word on their paper using their ink/paper/rules  $\Rightarrow$  this is our base for a Turing machine (TM) and the reason to bring it up is to ask: what can it not compute?
    - TM solve EPs  $\Rightarrow$  Church-Turing Thesis: Turing machines can compute anything that algorithms can and vice versa
    - example:  $x \in \text{PRIM}$ ? Let  $M_{\text{PRIM}}$  be a TM for PRIM  $\rightarrow x \in \text{PRIM} \Rightarrow M_{\text{PRIM}}$  answers YES,  $x \notin \text{PRIM} \Rightarrow M_{\text{PRIM}}$  answers NO
    - Turing's Theorem: There are EPs unsolvable by TM (undecidable)  $\Rightarrow$  and here is where our DM knowledge comes in handy: Proof: Consider all TMs  $M_1, M_2, M_3, \dots$  and all words within a fixed alphabet  $w_1, w_2, w_3, \dots$  each TM solves an EP for each word and answers with YES/NO  $\rightarrow$
- |          | $w_1$ | $w_2$ | $w_3$ | $w_4$ | $w_5$ | $w_6$ | $\dots$ |
|----------|-------|-------|-------|-------|-------|-------|---------|
| $M_1$    | Y     | N     | N     | Y     | Y     | Y     | $\dots$ |
| $M_2$    | N     | N     | Y     | N     | N     | N     |         |
| $M_3$    | Y     | N     | N     | Y     | N     | Y     |         |
| $M_4$    | N     | N     | Y     | Y     | N     | N     |         |
| $\vdots$ |       |       |       |       |       |       |         |
- $\rightarrow$  now consider the EP DIAG  $\rightarrow w_i$  is in DIAG if  $M_i$  says NO
  - for  $w_i \rightarrow \text{DIAG} = \{w_2, w_3, \dots\}$
  - Consider the  $k$ -th TM,  $M_k \rightarrow$  does it have Y or N in  $(k, k)$ ?

→ if  $Y$ , it answers  $Y$  for  $W_k$ , meaning  $W_k \notin \text{DIAG}$ , if it answers  $N$ ,  $W_k \in \text{DIAG}$ , now assume  $M_k$  is a TM for  $\text{DIAG} \Rightarrow M_k$  has to answer  $Y$  to  $W_k$  ↯

- **Halting problem:** will some program with a given input terminate or run indefinitely?

- interesting example:

```
1 n = 4
2 p = 1
3 while True:
4     p = p + 1
5     if isprime(p):
6         q = n - p
7         if isprime(q):
8             print(n + " = " + p + "+" + q)
9             n = n + 2
10            p = 1
11        if p > n/2:
12            exit()
```

# Initialisierung  
# p und q sind prim und  $n = p + q$   
# Keine Primzahlen p und q mit  $n = p + q$  gefunden

→ whether this terminates is yet unknown → **Goldbach conjecture**, which states that all even numbers  $\geq 4$  are the sum of two primes

### Algorithm analysis:

assuming a decision problem is solvable by TM, how many steps

does it take? for PRIM, it's  $\in \mathbb{Z}^n$

→ Def.:  $f$  is in  $O(g)$  if for large  $n$ ,  $f$  grows at most constantly faster than  $g$

→ **Cobham-Edmond Thesis:** an algorithm is efficient if and only if it can be computed in <sup>(pol.)</sup> polynomial time

→ if an algorithm can solve a problem in  $O(n^k)$ , a TM can solve it in  $O(n^k)$   $\Rightarrow$  the class of polys. is robust

→ Def.: A problem is efficiently solvable if it is in the set  $P$  which has all problems that can efficiently be solved by TM  $\Rightarrow \text{PRIM} \in P$ ,  $\text{TSP (PV)}$  (Traveling Salesperson/Salesman depending on your politics)  $\notin P$

**NP-completeness:** A polynomial verifier does not need to compute the solution to a problem, but is given a "potential witness" (seq. of characters) with its input  $\rightarrow$  if the input resolves to YES, the witness proves it, if it resolves to NO, it is arbitrary  $\rightarrow$  TSP is verifiable in polynomial time, the witness is a seq. of vertices

Def.: The set/class NP contains all EPs that can efficiently be solved by a PC

→ as you most probably know, we don't know if  $P = NP$ , the assumption is  $P \neq NP$ , and you've also probably heard this a million times: if one polynomial algorithm is found for a NP-complete problem,  $P = NP$  (example: TSP)

Def.: A problem  $A$  in NP is NP-complete if an efficient algorithm for  $A$  allows for the efficient solution of all problems in NP

**Cook's Theorem:** There exists an NP-complete EP

**Karp's Theorem:** There exist 21 more NP-complete EP (there are actually thousands without an efficient TM)

**Polynomial Time Reductions:** Let  $A, B \in \text{NP}$ . We know  $A$  is NP-complete  $\rightarrow$  if  $A$  solvable efficiently, so are all others. Procedure: show that if  $B$  solvable efficiently,  $A$  is too  $\Rightarrow$  if  $B$  solvable efficiently, all others are too

$\Rightarrow B$  is NP-complete

→ We reduce solving  $A$  to solving  $B \Rightarrow$  Polynomial time reduction

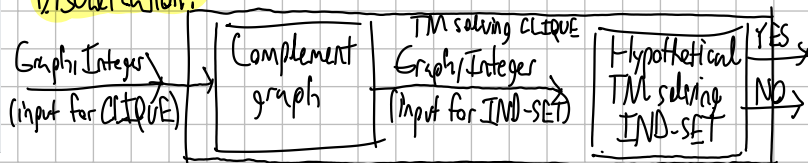
Example:

**Clique**: set of vertices (verts.) in a graph that are all connected pairwise, **Independent Set**: set of verts. of which none are connected  $\Rightarrow$  their EPs **CLIQUE** =  $\{(y, k) \mid y \text{ graph with clique of size } k\}$  and **IND-SET** =  $\{(y, k) \mid y \text{ graph with ind. set of size } k\}$   $\rightarrow$  procedure: both problems are NP-complete  $\rightarrow$  reduce CLIQUE to IND-SET

**Obs.:** The complement of a graph with a clique of size  $k$  has an ind. set of size  $k$



**Visualization:**



**Code:**

```
def solve_CLIQUE(G, k):
    for i in range(len(G)):
        for j in range(len(G[0])):
            if G[i][j] == 0:
                G[i][j] = 1
            else:
                G[i][j] = 0
    return solve_IS(G, k)

print(solve_CLIQUE([1,1,1], [1,0,1], [1,0,1], 2))
```

**Def.:** Search problem (SP) means not just to decide whether a solution exists but also to output said solution  $\rightarrow$  EPs can be expanded to SPs at varying degrees of solvability

Preface done, now we're doing real definitions: (Topic: Alphabets/Words/Languages)

**Def. 2.1** (from the book Theoretische Informatik which this course follows closely): An **alphabet**  $\Sigma$  is a finite, non-empty set of symbols, **examples:**  $\Sigma_{\text{bool}} = \{0, 1\}$ ,  $\Sigma_{\text{ASCII}} = \{a, b, c, \dots, z\}$ ,  $\Sigma_{\text{logic}} = \{0, 1, x, (, ), \vee, \wedge, \neg\}$

**Def. 2.2:** Let  $\Sigma$  be an alphabet. A **word** over  $\Sigma$  is a finite (possibly empty) seq. of letters from  $\Sigma$ . The empty word  $\lambda$  is the empty set of letters. The **length**  $|w|$  of a word is the length of the word as a sequence, meaning the amount of appearances of letters in  $w$ .  $\Sigma^*$  is the set of all words over  $\Sigma$ ,  $\Sigma^+ = \Sigma^* - \{\lambda\}$

$\rightarrow$  **example:** the seq. 0, 1, 0, 0, 1, 1 is a word over  $\Sigma_{\text{bool}}$  and  $\Sigma_{\text{keystroke}}$  (set of all keyboard characters),  $|0, 1, 0, 0, 1, 1| = 6$ ,  $\lambda$  is a word over all alphabets,  $|\lambda| = 0$

$\rightarrow$  A word  $x = x_1 x_2 \dots x_n \in \Sigma_{\text{bool}}$  can be seen as the bin. representation of the number  $\text{Number}(x) = \sum_{i=1}^n x_i \cdot 2^{n-i}$ , likewise, for ms (No, let  $\text{bin}(n)$  be the bin. representation without leading zeros with  $\text{bin}(0) = \lambda$ .

**Def. 2.3:** The **concatenation** (for an alphabet  $\Sigma$ ) is an image  $\text{Kon}: \Sigma^* \times \Sigma^* \rightarrow \Sigma^*$ , s.t.  $\text{Kon}(x, y) = x \cdot y = xy$  for all  $x, y \in \Sigma^*$ .

$\rightarrow$  **example:**  $x = 01, y = 11 \rightarrow xy = 0111, x = 110, y = \lambda, xy = yx = 110$

**Obs. 2.1:**  $\text{Kon}$  over  $\Sigma$  is an **assoc. op.** over  $\Sigma^*$ , as  $\text{Kon}(v, \text{Kon}(u, w)) = v \cdot (u \cdot w) = (v \cdot u) \cdot w = \text{Kon}(\text{Kon}(v, u), w) \rightarrow (\Sigma^*, \text{Kon})$  is a **monoid** with neutral element  $\lambda$

**Obs. 2.2:**  $\forall x, y \in \Sigma^* : |xy| = |x \cdot y| = |x| + |y|$

**Def. 2.4:** for a word  $w = a_1 \dots a_n$ , with  $a_i \in \Sigma$  for  $i \in \{1, \dots, n\}$ ,  $w^R = a_n a_{n-1} \dots a_1$  (Reversal of  $w$ )

**Def. 2.5:** Let  $\Sigma$  be an alphabet. For all  $x \in \Sigma^*$  and  $i \in \mathbb{N}$ , we define the  **$i$ -th iteration**  $x^i$  of  $x$  as  $x^0 = \lambda, x^1 = x, x^i = x x^{i-1}$ , **example:**  $\text{Kon}(aabbba, aaaaaa) = aabbbaaaaaa = a^2 b^2 a^6 = a^2 b^2 (aa)^3$

**Def. 2.6:** Let  $v, w \in \Sigma^*$  for an alphabet  $\Sigma$ .  $v$  is a **partial word** of  $w \Leftrightarrow \exists x, y \in \Sigma^* : w = vx y, v$  is a **prefix** of  $w \Leftrightarrow$

$\exists y \in \Sigma^+ : w = vy, v$  is a suffix of  $w \Leftrightarrow \exists x \in \Sigma^+ : w = xv, v \neq \lambda$  is a real partial word of  $w \Leftrightarrow v \neq \lambda$  and  $v$  is a partial word of  $w$

Def. 2.7. Let  $x \in \Sigma^*$  and  $a \in \Sigma$ . Then,  $|x|_a$  is the amount of occurrences of  $a$  in  $x$ .

Def. 2.8: Let  $\Sigma = \{s_1, s_2, \dots, s_m\}, m \geq 1$ , an alphabet and  $s_1 < s_2 < \dots < s_m$  an order on  $\Sigma$ . The canonical order on  $\Sigma^*$  for  $u, v \in \Sigma^*$  is:  $u < v \Leftrightarrow |u| < |v| \vee |u| = |v| \wedge u = x \cdot s_i \cdot u' \wedge v = x \cdot s_j \cdot v'$  for some  $x, u', v' \in \Sigma^*$  and is;

Def. 2.9. A language  $L$  over an alphabet  $\Sigma$  is a subset of  $\Sigma^*$ . The complement  $L^c$  of the language  $L$  w.r.t.  $\Sigma$  is the language  $\Sigma^* - L$ .  $L_\emptyset = \emptyset$  is the empty language,  $L_\lambda = \{\lambda\}$  is the one-element language, only consisting of the empty word. If  $L_1$  and  $L_2$  are languages over  $\Sigma$ ,  $L_1 \cdot L_2 = L_1 L_2 = \{uv \mid u \in L_1 \text{ and } v \in L_2\}$  is the concatenation of  $L_1$  and  $L_2$ . If  $L$  is a language over  $\Sigma$ , we define  $L^0 = L_\lambda$  and  $L^{i+1} = L^i \cdot L$  for all  $i \in \mathbb{N}$ ,  $L^* = \bigcup_{i \in \mathbb{N}} L^i$  and  $L^+ = \bigcup_{i \in \mathbb{N} \setminus \{0\}} L^i = L \cdot L^*$ , where  $L^*$  is known as Kleene's star over  $L$ .

for  $\Sigma = \{a, b\}$ :  $L_1 = \emptyset$ ,  $L_2 = \{\lambda\}$ ,  $L_3 = \{\lambda, ab, abab\}$ ,  $L_4 = \Sigma^+ = \{\lambda, a, b, aa, \dots\}$ ,  $L_5 = \Sigma^+ = \{\lambda, b, ba, \dots\}$ ,  $L_6 = \{a\}^* = \{\lambda, a, aa, aaa, \dots\} = \{a^i \mid i \in \mathbb{N}\}$ ,  $L_7 = \{a^p \mid p \text{ prime}\}$ ,  $L_8 = \{a^i b^{2i} a^i \mid i \in \mathbb{N}\}$ ,  $L_9 = \Sigma$ ,  $L_{10} = \Sigma^3 = \{aaa, aab, aba, \dots\}$  (these are all examples)  $\rightarrow$  the set of all grammatically correct German texts is a language over  $\Sigma_{\text{German}}$

Lemma 2.1. Let  $L_1, L_2, L_3$  be languages over an alphabet  $\Sigma$ . Then  $L_1 L_2 \cup L_1 L_3 = L_1 (L_2 \cup L_3)$

Proof: first, show " $\subseteq$ ":  $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\} \subseteq \{xy \mid x \in L_1, y \in L_2 \cup L_3\} = L_1 (L_2 \cup L_3) \rightarrow L_1 L_2 \subseteq L_1 (L_2 \cup L_3)$   
 $\rightarrow$  analogously,  $L_1 L_3 \subseteq L_1 (L_2 \cup L_3) \Rightarrow L_1 L_2 \cup L_1 L_3 \subseteq L_1 (L_2 \cup L_3)$   
 $\rightarrow$  now, show  $L_1 (L_2 \cup L_3) \subseteq L_1 L_2 \cup L_1 L_3$  (" $\supseteq$ "); Let  $x \in L_1 (L_2 \cup L_3)$ . Then,  $x \in \{yz \mid y \in L_1, z \in L_2 \cup L_3\}$  (def. concat.)  
 $\Rightarrow \exists y \in L_1 \exists z \in L_2 \cup L_3$  s.t.  $x = yz \Rightarrow \exists y \in L_1 \wedge (\exists z \in L_2 \vee \exists z \in L_3)$  s.t.  $x = yz$  (def.  $\cup$ )  $\Leftrightarrow (\exists y \in L_1 \exists z \in L_2 : x = yz) \vee (\exists y \in L_1 \exists z \in L_3 : x = yz)$  (dist.)  $\Leftrightarrow (x \in \{yz \mid y \in L_1, z \in L_2\}) \vee (x \in \{yz \mid y \in L_1, z \in L_3\})$   
 $\Leftrightarrow x \in L_1 L_2 \cup L_1 L_3$

Lemma 2.2: Let  $L_1, L_2, L_3$  be languages over an alphabet  $\Sigma$ . Then,  $L_1 (L_2 \cap L_3) \subseteq L_1 L_2 \cap L_1 L_3$

Proof: Let  $x \in L_1 (L_2 \cap L_3)$ , equiv. to  $x \in \{yz \mid y \in L_1, z \in L_2 \cap L_3\}$  (def. concat.)  $\Leftrightarrow \exists y, z \in \Sigma^+, y \in L_1 \wedge (z \in L_2 \cap L_3)$  s.t.  $x = yz$  (def.  $\cap$ )  $\Leftrightarrow \exists y, z \in \Sigma^+, (y \in L_1 \wedge z \in L_2) \wedge (y \in L_1 \wedge z \in L_3)$   $\wedge x = yz \Rightarrow \exists y, z \in \Sigma^+, (yz \in L_1 L_2) \wedge (yz \in L_1 L_3) : x = yz$  (def. concat.)  $\Leftrightarrow x \in L_1 L_2 \cap L_1 L_3$  (def.  $\cap$ )

Lemma 2.3. There exist  $U_1, U_2, U_3 \in (\Sigma_{\text{bool}})^*$  s.t.  $U_1 (U_2 \cap U_3) \neq U_1 U_2 \cap U_1 U_3$ .

Proof: Let  $U_2 = \{0\}$ ,  $U_3 = \{10\} \Rightarrow U_2 \cap U_3 = \emptyset$ ,  $U_1 (U_2 \cap U_3) = \emptyset \rightarrow$  now pick  $U_1$  s.t.  $U_1 U_2 \cap U_1 U_3$  not empty  
 $\Rightarrow U_1 = \{\lambda, 1\} \Rightarrow U_1 U_2 = \{\emptyset, 10\}$ ,  $U_1 U_3 = \{\emptyset, 110\} \Rightarrow U_1 U_2 \cap U_1 U_3 = \{\emptyset, 10\}$

Example 2.1: We want to prove  $\{a\}^* \{b\}^* = \{a^i b^j \mid i, j \in \mathbb{N}\}$ . First, show  $\{a\}^* \{b\}^* \subseteq \{a^i b^j \mid i, j \in \mathbb{N}\}$ :

Let  $x \in \{a\}^* \{b\}^* \Rightarrow x = yz$  with  $y \in \{a\}^* \wedge z \in \{b\}^*$  (def. concat.)  $\Rightarrow x = yz$  with  $(\exists k \in \mathbb{N} : y = a^k) \wedge (\exists m \in \mathbb{N} :$

$z \in \Sigma b^* \Sigma^m$  (def. Kleene's star)  $\Leftrightarrow x = yz$  with  $(\exists k \in \mathbb{N}: y = c^k) \wedge (\exists m \in \mathbb{N}: z = b^m) \Leftrightarrow \exists k, m \in \mathbb{N}$  with  $x = c^k b^m$

$\Rightarrow x \in \{a^i b^j \mid i, j \in \mathbb{N}\}$ . Now, prove  $\{a^i b^j \mid i, j \in \mathbb{N}\} \subseteq \{a^* b^*\}$ . Let  $x \in \{a^i b^j \mid i, j \in \mathbb{N}\}$ . Then  $x = a^i b^j$  for some

$i, j \in \mathbb{N} \Rightarrow x \in \Sigma a^* \Sigma b^*$ , as  $a^i \in \Sigma a^*$ ,  $b^j \in \Sigma b^*$

**Def. 2.10:** Let  $\Sigma_1$  and  $\Sigma_2$  be two arbitrary alphabets. A **homomorphism** from  $\Sigma_1^*$  to  $\Sigma_2^*$  is any function  $h: \Sigma_1^* \rightarrow \Sigma_2^*$  with i)  $h(\lambda) = \lambda$ , ii)  $h(uv) = h(u) \cdot h(v)$  for all  $u, v \in \Sigma_1^*$